K J SOMAIYA · OPERATING SYSTEMS · AI USE-CASE ASSIGNMENT

AI-Based Irrigation Advisory System for Sugarcane

Use case **KJS-AGR-01** (KIAAR · Godavari Biorefineries) — a Linux prototype integrating four Operating Systems experiments

Exp 4 · System calls | Exp 6 · SJF & RR | Exp 7 · Producer–Consumer | Exp 9 · Banker's algorithm

Student name	Roll number	Batch
Param Sangani	16010424108	B2
Ankit Vishwakarma	16010424120	B2
Aarya Thota	16010424115	B2

Source code: https://github.com/ParamDesperado/OS-MiniProject-Irrigation-Advisory

Platform: Ubuntu 24.04.5 LTS (GitHub Codespaces), Linux 6.8, gcc 13.3, Python 3

Contents

1. Selected use case and experiments
2. Problem statement, objectives and prototype scope
3. Dataset and synthetic-data generation
4. Architecture, tools and AI component
5. Experiment 4 – File and process handling using system calls
6. Experiment 6 – Shortest Job First and Round Robin scheduling
7. Experiment 7 – Producer–Consumer using semaphores and mutex
8. Experiment 9 – Deadlock avoidance using the Banker's algorithm
9. Testing and performance comparison
10. Limitations, data privacy and improvements
11. Conclusion, references and AI-assistance acknowledgement

1. Selected Use Case and Experiments

Use case	**KJS-AGR-01** — Irrigation Advisory System for Sugarcane Crop using AI and Sensor-based Technology. Collaborators: K J Somaiya Institute of Applied Agricultural Research (KIAAR), Sameerwadi, and Godavari Biorefineries Ltd.
Experiments selected	**4, 6, 7 and 9**. All four are from Experiments 4–10. The requirement is at least two.
Prototype scope (from assignment)	Process soil moisture, weather and plot records to generate plot-wise irrigation recommendations. Sensor-reading, prediction and advisory tasks run as cooperating processes on Linux.

Exp	OS concept	Connection to the use case
4	fork, exec, wait, open/read/write/close	The application itself. A parent process forks and execs one child per workflow stage: acquisition (2 children), AI prediction (1 child per village, in parallel) and advisory generation. All data files are read and written with raw system calls.
6	SJF and Round Robin	The 8 processes of the pipeline are scheduled using their **measured CPU bursts**, which the parent records with `wait4()`. The experiment shows which policy suits a single-core field gateway.
7	Producer–Consumer with semaphores and mutex	Village sensor-gateway threads stream soil-moisture readings into a bounded buffer. Analytics threads consume them and update the shared plot-water table. The race condition is shown and then prevented.
9	Banker's algorithm	Villages compete for shared pump sets, 3-phase power slots, canal valves and fertigation units. Each village's maximum claim is the demand computed by the advisory stage.

2. Problem Statement, Objectives and Scope

Problem. Sugarcane is a water-intensive crop. In the KIAAR/GBL command area of North Karnataka, about 18,000–25,000 farmers decide when to irrigate from experience rather than from soil, crop and weather data.

The result is over-irrigation or delayed irrigation, which wastes water and electricity and causes yield loss from moisture stress. Field staff also cannot monitor thousands of spread-out plots by hand.

Objectives of this prototype.

1. Ingest soil-moisture sensor readings, a 3-day weather forecast and geo-fenced plot records, and validate them by rejecting missing, out-of-range and malformed data.
2. Predict, for every plot, the **irrigation depth** needed over the next 3 days and the **probability of water stress** using trained ML models, with uncertainty bounds.
3. Produce farmer-friendly, prioritised advisories in English and Hindi (irrigate today / tomorrow / within 3 days / skip because rain is expected), along with pump-hours.
4. Run the workflow as cooperating Linux processes and threads, and use OS techniques for scheduling, safe sharing of sensor data and deadlock-free allocation of irrigation resources.

Scope. The project is a software prototype on Linux with synthetic data. IoT hardware, satellite imagery, the mobile app and the LLM advisory layer of the full use case are outside the scope of this assignment. They are discussed as future work.

3. Dataset and Synthetic-Data Generation

Real farmer data is not available to students and is personal data under the DPDP Act. All data is therefore **synthetic and clearly labelled**. It is generated by `scripts/generate_data.py` using only the Python standard library with a fixed seed (42), so every run reproduces it exactly.

File	Records	How it is generated
plots.csv	600 plots, 5 villages	Villages: Sameerwadi, Mudhol, Jamkhandi, Rabkavi and Terdal (unequal sizes 180/138/120/90/72). Each plot gets a random anonymised farmer token, a soil type (black / loam / red, with field capacity and wilting point), area 0.4–3.5 ha, crop age 10–360 days, drip or furrow irrigation, and a pump rating of 3.7, 5.5 or 7.5 kW.
sensor_readings.csv	100,800	14 days of readings every 2 h per plot. Soil moisture follows a drying curve whose rate depends on the crop coefficient, plus irrigation refills and sensor noise. **About 1.5 % of rows are deliberately faulty**: 524 missing, 495 out-of-range and 523 malformed.
weather_forecast.csv	15	A 3-day forecast per village with distinct profiles: Jamkhandi has rain expected, Rabkavi has heat (37 °C).
training_history.csv	5,000	Labelled samples. The labels come from a FAO-56 style soil water balance (depletion + crop ET − effective rain, with heat stress and a soil-specific refill target) plus Gaussian noise. The stress label is drawn from a logistic function of relative available water.

4. Architecture, Tools and AI Component

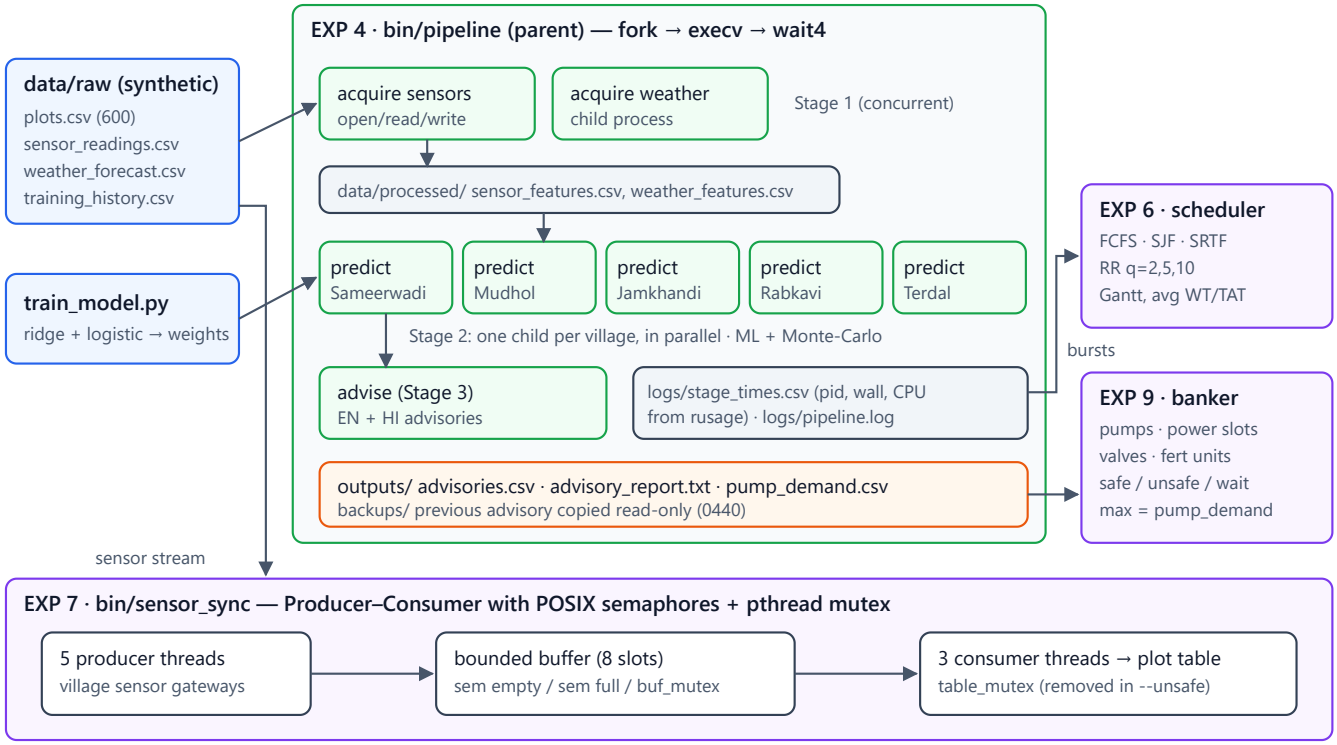


Figure 1 – Architecture: application data flow and the OS components used (processes, files, threads, shared buffer, synchronisation, resource allocation)

Tools used

Item	Used
Linux distribution	Ubuntu 24.04.5 LTS, kernel 6.8.0 (GitHub Codespaces, 2 vCPU). The same code also runs on WSL, a VM or a Raspberry Pi.
Languages / compiler	C11 with gcc 13.3 (`-Wall -Wextra -O2`, zero warnings), POSIX threads and semaphores. Python 3 (standard library only) for data generation and training. Bash for scripts.
Libraries	glibc, libpthread, libm. matplotlib is used only to draw the report charts.
Build / run	Makefile (`make`, `make test`), `scripts/run_all.sh`, and GitHub Actions CI that builds and tests on Ubuntu after every push.

```
codespace@ubuntu-24.04:~/OS-MiniProject$ grep PRETTY_NAME /etc/os-release; uname -sr; gcc --version | head -1; make
PRETTY_NAME="Ubuntu 24.04.5 LTS"
Linux 6.8.0-1064-azure
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
mkdir -p bin
gcc -Wall -Wextra -Wno-format-truncation -O2 -std=c11 -Iinclude -c src/common.c -o bin/common.o
gcc -Wall -Wextra -Wno-format-truncation -O2 -std=c11 -Iinclude src/pipeline.c bin/common.o -o bin/pipeline -lm
gcc -Wall -Wextra -Wno-format-truncation -O2 -std=c11 -Iinclude src/acquire.c bin/common.o -o bin/acquire -lm
gcc -Wall -Wextra -Wno-format-truncation -O2 -std=c11 -Iinclude src/predict.c bin/common.o -o bin/predict -lm
gcc -Wall -Wextra -Wno-format-truncation -O2 -std=c11 -Iinclude src/advise.c bin/common.o -o bin/advise -lm
gcc -Wall -Wextra -Wno-format-truncation -O2 -std=c11 -Iinclude src/scheduler.c bin/common.o -o bin/scheduler -lm
gcc -Wall -Wextra -Wno-format-truncation -O2 -std=c11 -Iinclude -pthread src/sensor_sync.c bin/common.o -o bin/sensor_sync -lm
gcc -Wall -Wextra -Wno-format-truncation -O2 -std=c11 -Iinclude src/banker.c -o bin/banker
```

Screenshot 1 – Linux environment and a clean build of all programs with gcc

AI / ML component

Ten agronomic features are engineered from the raw data. They are the same in training (Python) and inference (C): soil-water depletion in the root zone, 3-day crop evapotranspiration (Hargreaves ET_0 × FAO-56 sugarcane K_c by growth stage), effective rain (amount × probability), heat index above 36 °C, total available water, relative available water, K_c, T_{max}, forecast rain, and a black-soil interaction term.

- **Irrigation-depth regressor**: ridge linear regression (closed-form normal equations).
- **Water-stress classifier**: logistic regression trained with IRLS (Newton's method).
- **Uncertainty**: for each plot, a 4,000-draw Monte-Carlo ensemble over rain occurrence and amount, temperature error and sensor noise gives a P10–P90 range for depth. This ensemble is also what makes the per-village prediction processes CPU-heavy, which gives Experiment 6 realistic bursts.
- **Rule-based baseline** (clearly identified): a FAO-56 water balance, depth = depletion + 3-day ETc − forecast rain. It is used for comparison and as an automatic fallback if the model file is missing.

```
codespace@ubuntu-24.04:~/OS-MiniProject$ python3 scripts/train_model.py
Model evaluation on held-out test set (20%, synthetic data)
train samples = 4000, test samples = 1000

Irrigation depth (mm)          MAE     RMSE    R^2
  ML ridge regression          3.51     4.66   0.986
  Rule baseline (FAO-56)      24.93    29.59   0.430

Water-stress classifier       Acc    Prec   Recall  F1
  ML logistic regression    0.861  0.866  0.847  0.857
  Rule (RAW < 50%)          0.858  0.855  0.855  0.855

[train_model] weights written to models/irrigation_model.txt
```

Screenshot 2 – Model training and evaluation on the 20 % held-out test set (synthetic data)

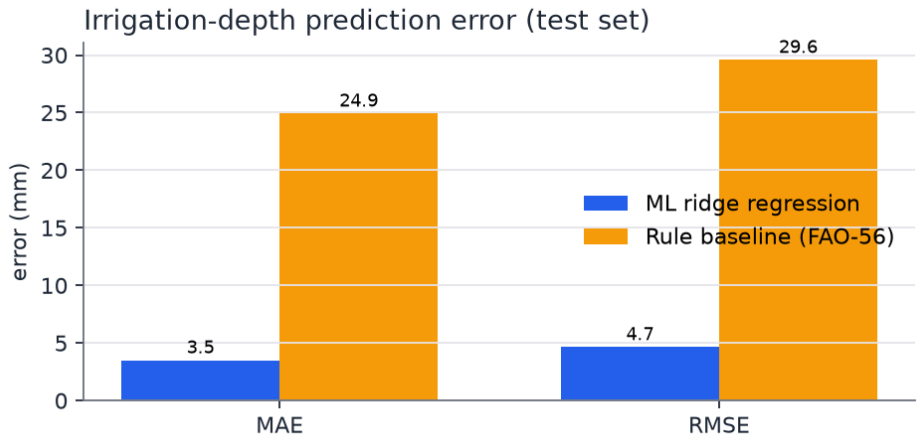


Figure 2 – The ML model cuts depth-prediction error from 24.9 mm to 3.5 mm MAE (R^2 0.986 vs 0.430)

Interpretation. The regressor clearly beats the rule baseline because it learns effects the fixed rule ignores: rain probability, the effective-rain fraction, heat stress and the management-allowed depletion. The stress classifier is only marginally better than the simple threshold rule (F1 0.857 vs 0.855). A threshold on relative

available water is already a strong signal on this data, so we report the result honestly rather than over-claim it.

5. Experiment 4 – File and Process Handling using System Calls

Aim. Write a program in which a parent process creates child processes with `fork()`, `exec()` and `wait()` for the stages of the workflow, and reads and writes use-case data files with `open()`, `read()`, `write()` and `close()`.

Use-case connection. This program *is* the irrigation-advisory application. `bin/pipeline` is the parent. Its children exec three separate programs:

Stage	Child process(es)	Work done	System calls
1 Acquisition	acquire sensors, acquire weather (concurrent)	Validate 100,800 sensor rows (rejects missing, out-of-range and malformed rows) and aggregate features per plot. Aggregate the forecast per village.	open, read (8 KB buffered line reader), write, close
2 Prediction	predict <village> × 5 (concurrent)	ML inference + Monte-Carlo ensemble for that village's plots	open, read, write, close
3 Advisory	advise	Merge predictions and write the advisories, report and pump demand	open(O_CREAT|O_TRUNC, 0640), write, close
Parent	pipeline	Creates directories, backs up the previous advisory as read-only, forks and execs the stages, reaps children with `wait4()` to get exit status and CPU usage, and aborts if a stage fails	fork, execv, wait4, mkdir, open, read, write, close, getpid

Key program logic (src/pipeline.c)

```
pid_t pid = fork();
if (pid == 0) {                       /* child: become the stage program        */
    execv(argv[0], argv);             /* e.g. bin/predict Mudhol ...            */
    _exit(127);                       /* only reached if exec fails             */
}
/* parent: reap every child of the stage, collect exit code + CPU time       */
pid_t done = wait4(-1, &status, 0, &ru);
cpu_ms = ru.ru_utime + ru.ru_stime;   /* -> logs/stage_times.csv (used by Exp 6) */
if (WEXITSTATUS(status) != 0)  ->  abort workflow (no stage consumes bad data)
```

Commands

```
make                     # builds bin/pipeline bin/acquire bin/predict bin/advise ...
./bin/pipeline           # run the application
ps -eo pid,ppid,ni,stat,comm | grep -E 'pipeline|predict'   # observe the process tree
cat logs/stage_times.csv  # per-process wall and CPU time
```

```
codespace@ubuntu-24.04:~/OS-MiniProject$ ./bin/pipeline
═══ Irrigation advisory pipeline (KJS-AGR-01)  parent pid=4603 ═══
plots.csv: 600 plots in 5 villages

[Stage 1] data acquisition
  fork → pid 4609   exec acquire_sensors             (parent pid 4603)
  fork → pid 4610   exec acquire_weather             (parent pid 4603)
[acquire   pid=4610] weather: 5 villages aggregated, 0 invalid rows rejected
  wait ← pid 4610   acquire_weather            exit=0  wall=   1.28 ms  cpu=   1.08 ms
[acquire   pid=4609] sensors: 100800 rows read, 99258 valid, rejected: 524 missing, 495 out-of-range, 523 malformed → 600 plots
  wait ← pid 4609   acquire_sensors            exit=0  wall=  35.85 ms  cpu=  34.49 ms

[Stage 2] AI prediction - 5 village processes in parallel
  fork → pid 4611   exec predict_Sameerwadi          (parent pid 4603)
  fork → pid 4612   exec predict_Mudhol              (parent pid 4603)
  fork → pid 4613   exec predict_Jamkhandi           (parent pid 4603)
  fork → pid 4614   exec predict_Rabkavi             (parent pid 4603)
  fork → pid 4615   exec predict_Terdal              (parent pid 4603)
[predict   pid=4613] Jamkhandi: 120 plots predicted, 0 sensor-offline, 4000 MC draws/plot, 166.1 ms (ML model)
  wait ← pid 4613   predict_Jamkhandi          exit=0  wall= 170.34 ms  cpu= 121.89 ms
[predict   pid=4615] Terdal: 72 plots predicted, 0 sensor-offline, 4000 MC draws/plot, 185.2 ms (ML model)
  wait ← pid 4615   predict_Terdal             exit=0  wall= 192.26 ms  cpu=  76.03 ms
[predict   pid=4614] Rabkavi: 90 plots predicted, 0 sensor-offline, 4000 MC draws/plot, 220.4 ms (ML model)
  wait ← pid 4614   predict_Rabkavi            exit=0  wall= 229.00 ms  cpu=  92.41 ms
[predict   pid=4612] Mudhol: 138 plots predicted, 0 sensor-offline, 4000 MC draws/plot, 289.4 ms (ML model)
  wait ← pid 4612   predict_Mudhol             exit=0  wall= 295.81 ms  cpu= 140.91 ms
[predict   pid=4611] Sameerwadi: 180 plots predicted, 0 sensor-offline, 4000 MC draws/plot, 320.8 ms (ML model)
  wait ← pid 4611   predict_Sameerwadi         exit=0  wall= 329.01 ms  cpu= 175.58 ms

[Stage 3] advisory and report generation
  fork → pid 4617   exec advise_report               (parent pid 4603)
[advise    pid=4617] 600 advisories: 244 today, 95 tomorrow, 230 within 3 days, 31 none, 0 sensor-offline
  wait ← pid 4617   advise_report              exit=0  wall=   8.15 ms  cpu=   7.97 ms

PIPELINE COMPLETE: 8 child processes, total wall 375.90 ms, total child CPU 650.37 ms
outputs: outputs/advisories.csv, outputs/advisory_report.txt, outputs/pump_demand.csv; timings: logs/stage_times.csv
```

Screenshot 3 – The parent forks 8 children in 3 stages. The 5 prediction processes run in parallel, and every child is reaped with its exit code and CPU time

```
codespace@ubuntu-24.04:~/OS-MiniProject$ ./bin/pipeline -o /tmp/o -l /tmp/l >/dev/null 2>&1 & sleep 0.1; ps -o pid,ppid,stat,args --ppid $! -p $! | cut -c1-80; wait
    PID    PPID STAT COMMAND
   4627    4625 S<   ./bin/pipeline -o /tmp/o -l /tmp/l
   4632    4627 R<   bin/predict Sameerwadi data/raw/plots.csv data/processed/se
   4633    4627 R<   bin/predict Mudhol data/raw/plots.csv data/processed/sensor
   4634    4627 R<   bin/predict Jamkhandi data/raw/plots.csv data/processed/sen
   4635    4627 R<   bin/predict Rabkavi data/raw/plots.csv data/processed/senso
   4636    4627 R<   bin/predict Terdal data/raw/plots.csv data/processed/sensor
```

Screenshot 4 – ps during a run: the five predict processes all have PPID 4627, the pipeline parent

codespace@ubuntu-24.04: ~/OS-MiniProject-Irrigation-Advisory

```
task,stage,pid,start_ms,end_ms,wall_ms,cpu_ms,exit_code
acquire_sensors,1,4609,2.543,38.392,35.849,34.486,0
acquire_weather,1,4610,2.704,3.984,1.280,1.084,0
predict_Sameerwadi,2,4611,38.481,367.489,329.008,175.580,0
predict_Mudhol,2,4612,38.629,334.436,295.807,140.912,0
predict_Jamkhandi,2,4613,38.794,209.137,170.343,121.888,0
predict_Rabkavi,2,4614,38.972,267.975,229.003,92.413,0
predict_Terdal,2,4615,39.179,231.442,192.263,76.032,0
advise_report,3,4617,367.552,375.707,8.155,7.973,0
```

Output 1 – logs/stage_times.csv written by the parent with write(): measured burst times used in Experiment 6

```
codespace@ubuntu-24.04:~/OS-MiniProject$ cat outputs/advisory_report.txt
================================================================================
 SUGARCANE IRRIGATION ADVISORY REPORT  (KJS-AGR-01 prototype, SYNTHETIC DATA)
 Decision support only - final irrigation decision rests with farmer / field officer
================================================================================

Village     Plots  Today   Tmrw  3-day   None Offline     AI(m3)   Rule(m3)
Sameerwadi    180     68     27     81      4       0     290090     464063
Mudhol        138     57     24     56      1       0     222203     348282
Jamkhandi     120     34     16     49     21       0     116297     167380
Rabkavi        90     51     18     20      1       0     186223     280636
Terdal         72     34     10     24      4       0     137548     195715
TOTAL         600    244     95    230     31       0     952361    1456076

3-day water requirement, AI advisory vs rule-based schedule: 952361 m3 vs 1456076 m3 (34.6% less)

Top 15 most urgent plots
Plot   Village     Method  Stress   NetNeed  GrossApp  PumpHrs Action
P0488  Rabkavi     furrow    0.99  126.5 mm  125.0 mm     40.8 IRRIGATE_TODAY
P0059  Sameerwadi  furrow    0.99  129.0 mm  125.0 mm     99.3 IRRIGATE_TODAY
P0122  Sameerwadi  furrow    0.99  133.0 mm  125.0 mm     39.3 IRRIGATE_TODAY
P0249  Mudhol      furrow    0.99  110.5 mm  125.0 mm     52.0 IRRIGATE_TODAY
P0157  Sameerwadi  furrow    0.99  120.0 mm  125.0 mm     41.3 IRRIGATE_TODAY
P0218  Mudhol      furrow    0.99  138.4 mm  125.0 mm     48.5 IRRIGATE_TODAY
P0503  Rabkavi     furrow    0.99  124.5 mm  125.0 mm    114.5 IRRIGATE_TODAY
P0178  Sameerwadi  furrow    0.98  123.0 mm  125.0 mm     30.5 IRRIGATE_TODAY
P0220  Mudhol      drip      0.98  125.0 mm   44.4 mm      6.7 IRRIGATE_TODAY
P0134  Sameerwadi  furrow    0.98  124.2 mm  125.0 mm     67.5 IRRIGATE_TODAY
P0212  Mudhol      furrow    0.98  123.5 mm  125.0 mm     41.3 IRRIGATE_TODAY
P0453  Rabkavi     drip      0.98  120.2 mm   44.4 mm     14.2 IRRIGATE_TODAY
P0514  Rabkavi     drip      0.98  120.0 mm   44.4 mm     12.7 IRRIGATE_TODAY
P0114  Sameerwadi  furrow    0.98  124.1 mm  125.0 mm     70.0 IRRIGATE_TODAY
P0185  Mudhol      furrow    0.98  121.8 mm  125.0 mm     81.1 IRRIGATE_TODAY

Sample farmer messages (one per action type)
  [IRRIGATE_TODAY] P0488 (Rabkavi)
    EN: High water-stress risk. Start irrigation today: run pump 40.8 h (5.1 days of power supply), 125 mm.
    HI: [illegible]
  [IRRIGATE_TOMORROW] P0171 (Sameerwadi)
    EN: Irrigate tomorrow morning: run pump 4.0 h (0.5 days of power supply), 44 mm.
    HI: [illegible]
  [IRRIGATE_WITHIN_3_DAYS] P0408 (Jamkhandi)
    EN: Irrigate within the next 3 days: run pump 32.2 h, 94 mm.
    HI: [illegible]
  [NO_IRRIGATION] P0024 (Sameerwadi)
    EN: Soil moisture is adequate. No irrigation needed for the next 3 days.
    HI: [illegible]
  [SKIP_RAIN_EXPECTED] P0354 (Jamkhandi)
    EN: Rainfall of about 33 mm is expected; irrigation is not required.
    HI: [illegible]
```

Screenshot 5 – outputs/advisory_report.txt generated by the advise child process, with farmer messages in English and Hindi

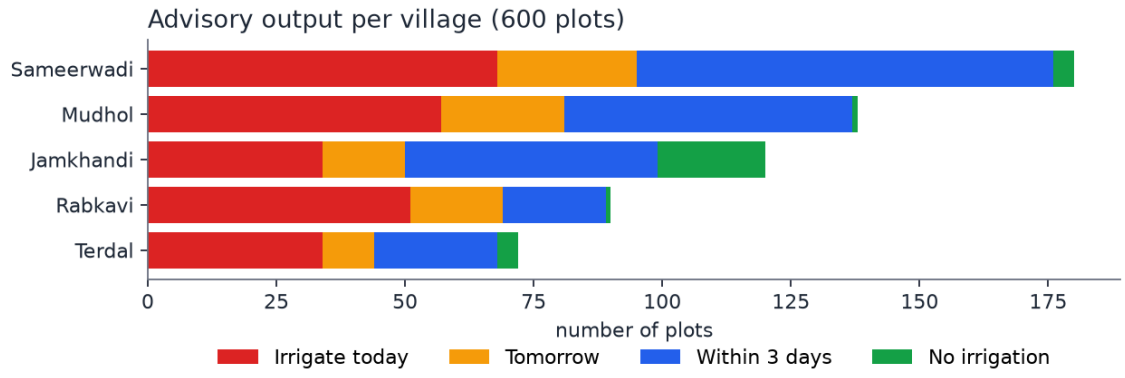


Figure 3 – Advisory decisions per village. Jamkhandi has the most 'no irrigation' plots because rain is forecast there

Results

- 8 child processes ran successfully. Total wall time was 376 ms against 650 ms of total child CPU time, so the parallel village processes overlap on the 2-core machine.
- 1,542 faulty sensor rows were detected and rejected, and all 600 plots received an advisory: 244 today, 95 tomorrow, 230 within 3 days and 31 none.
- The 3-day water requirement estimated by the AI advisory is **34.6 % lower** than the rule-based schedule (952,361 m^3 vs 1,456,076 m^3).
- Failure handling: if the sensor file is missing, Stage 1 exits with code 2 and the parent stops the workflow. A corrupt file gives exit code 3. A missing model makes the system fall back to the rule baseline (see Section 9).

Post-lab questions

Q1. What is the difference between fork() and exec()?
fork() creates a new process that is a copy of the caller, with a new PID and the same code and data. exec() replaces the code of the calling process with a new program and keeps the PID. The pipeline uses both: fork to create a child, then execv to turn the child into `acquire`, `predict` or `advise`.

Q2. Why must the parent call wait()?
To collect the child's exit status, so the parent knows whether a stage succeeded, and to remove the child's process-table entry. Without it, finished children stay as zombies. We use `wait4()` because it also returns the child's resource usage (CPU time).

Q3. Why use open/read/write instead of fopen/fprintf?
They are the kernel system-call interface itself. They give direct control over file descriptors, permissions (0640 for outputs, 0440 for backups) and flags such as O_TRUNC. The C library's stdio functions are buffered wrappers built on top of these calls.

Q4. What is an orphan process?
A child whose parent exits first. It is adopted by init/systemd. The pipeline always waits for every child of a stage before it moves on, so no orphans or zombies are left.

Outcome / conclusion. The advisory workflow runs as cooperating Linux processes created with fork/exec and synchronised by wait. Village-level parallelism and stage-failure isolation come directly from the process model. All file I/O uses raw system calls.

6. Experiment 6 – Shortest Job First and Round Robin Scheduling

Aim. Model the tasks of the use case as processes with burst times taken from the prototype. Implement SJF and RR, draw Gantt charts, compute average waiting and turnaround time, compare at least two time quanta, and justify the policy best suited to the use case.

Use-case connection and source of burst times. The 8 processes are the real tasks of one advisory cycle. The burst times are **measured, not assumed**. Each burst is the CPU time (user + system) of that child process, reported by `wait4()` rusage in Experiment 4 and stored in `logs/stage_times.csv`. The time unit is 1 tick = 1 ms. Two arrival scenarios are analysed:

- **Measured**: arrival = the fork offset measured by the pipeline (stage dependencies included).
- **Batch cycle**: all 8 tasks of a cycle are queued at t = 0 on a single-core field gateway (for example a Raspberry Pi).

Algorithms implemented in `src/scheduler.c`: FCFS (reference), SJF (non-preemptive), SRTF (preemptive SJF) and Round Robin with quanta 2, 5 and 10 ms. If no measurement log exists, documented estimates in `data/config/task_estimates.csv` are used and the program says so.

```
./bin/scheduler -a measured -t 1 -q 2,5,10     # measured arrivals and bursts
./bin/scheduler -a zero     -t 1 -q 2,5,10     # batch cycle: all arrive at t = 0
```

```
codespace@ubuntu-24.04:~/OS-MiniProject$ ./bin/scheduler -a zero -q 2,5,10 | sed -n "1,15p;/==== SJF/,/^ [0-9]/p"
EXPERIMENT 6 - CPU scheduling of irrigation-advisory tasks (KJS-AGR-01)
Source of burst times : MEASURED (wait4 rusage CPU time from Exp-4 pipeline)
Input file            : logs/stage_times.csv
Arrival mode          : zero (batch cycle)   |   1 tick = 1.00 ms

PID  Task                      arrival    CPU burst       AT       BT
P1   acquire_sensors            0.00ms     34.486ms        0       34
P2   acquire_weather            0.00ms      1.084ms        0        1
P3   predict_Sameerwadi         0.00ms    175.580ms        0      176
P4   predict_Mudhol             0.00ms    140.912ms        0      141
P5   predict_Jamkhandi          0.00ms    121.888ms        0      122
P6   predict_Rabkavi            0.00ms     92.413ms        0       92
P7   predict_Terdal             0.00ms     76.032ms        0       76
P8   advise_report              0.00ms      7.973ms        0        8

==================== SJF ====================
PID  Task                       AT     BT     CT    TAT     WT     RT
P1   acquire_sensors             0     34     43     43      9      9
P2   acquire_weather             0      1      1      1      0      0
P3   predict_Sameerwadi          0    176    650    650    474    474
P4   predict_Mudhol              0    141    474    474    333    333
P5   predict_Jamkhandi           0    122    333    333    211    211
P6   predict_Rabkavi             0     92    211    211    119    119
P7   predict_Terdal              0     76    119    119     43     43
P8   advise_report               0      8      9      9      1      1
Average WT = 148.75 ticks (148.75 ms)   Average TAT = 230.00 ticks (230.00 ms)   Avg RT = 148.75 ticks
Context switches = 7   Throughput = 12.3 tasks/s
Gantt chart (1 tick = 1.00 ms):
 |P2   |P8   |P1   |P7   |P6   |P5   |P4   |P3   |
 0     1     9     43    119   211   333   474   650
```

Screenshot 6 – Process table built from the measured CPU bursts, and SJF for the batch cycle: the shortest tasks (weather, report, sensors) run first

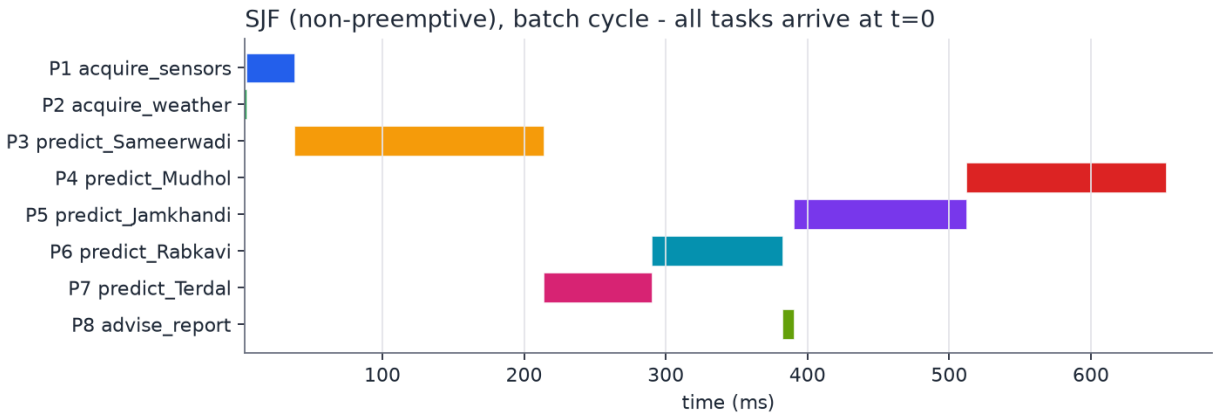


Figure 4 – Gantt chart, SJF (batch cycle)

codespace@ubuntu-24.04: ~/OS-MiniProject-Irrigation-Advisory

```
==================== RR_q5 ====================
PID  Task                      AT     BT     CT    TAT     WT     RT
P1   acquire_sensors            0     34    193    193    159      0
P2   acquire_weather            0      1      6      6      5      5
P3   predict_Sameerwadi         0    176    650    650    474      6
P4   predict_Mudhol             0    141    619    619    478     11
P5   predict_Jamkhandi          0    122    583    583    461     16
P6   predict_Rabkavi            0     92    496    496    404     21
P7   predict_Terdal             0     76    439    439    363     26
P8   advise_report              0      8     69     69     61     31
Average WT = 300.62 ticks (300.62 ms)   Average TAT = 381.88 ticks (381.88 ms)   Avg RT = 14.50 ticks
Context switches = 128   Throughput = 12.3 tasks/s
Gantt chart (1 tick = 1.00 ms):
 |P1   |P2   |P3   |P4   |P5   |P6   |P7   |P8   |P1   |P3   |P4   |P5   |
 0     5     6     11    16    21    26    31    36    41    46    51    56
   ... 105 more slices (full chart in gantt_RR_q5.csv) ...
 |P3   |P4   |P5   |P3   |P4   |P3   |P4   |P3   |P4   |P3   |P4   |P3   |
 571   576   581   583   588   593   598   603   608   613   618   619   650
```

Output 2 – Round Robin q = 5 ms (batch cycle); the full chart is in outputs/scheduling/gantt_RR_q5.csv

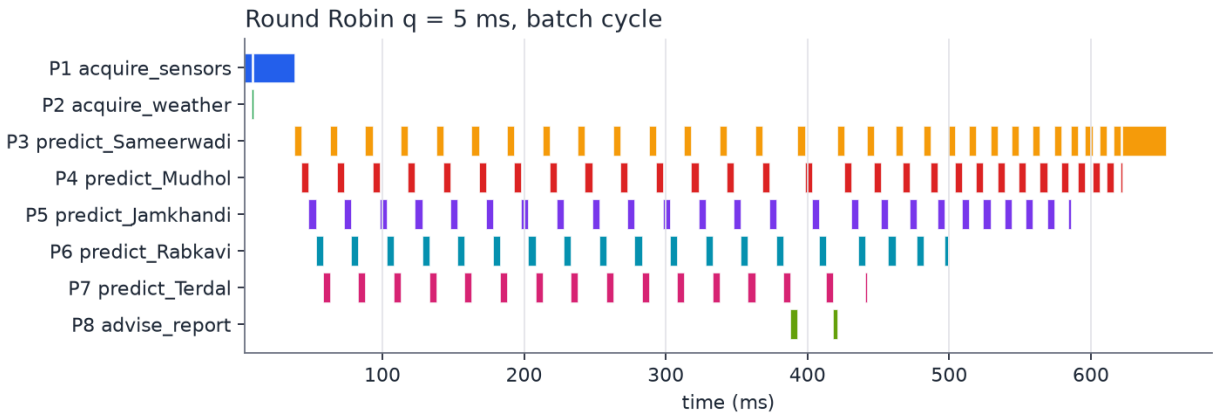


Figure 5 – Gantt chart, Round Robin q = 5 ms (batch cycle)

Comparison

```
codespace@ubuntu-24.04:~/OS-MiniProject$ ./bin/scheduler -a measured -q 2,5,10 | sed -n "/COMPARISON/,\$p"
==================== COMPARISON ====================
Algorithm    Avg WT(ms)  Avg TAT(ms)   Avg RT(ms)  Switches    Tasks/s
FCFS             221.25       302.50       221.25         7       12.3
SJF              158.12       239.38       158.12         7       12.3
SRTF             122.75       204.00        66.88         9       12.3
RR_q2            251.00       332.25         3.25       294       12.3
RR_q5            256.50       337.75         8.88       123       12.3
RR_q10           260.25       341.50        18.25        65       12.3

Lowest average waiting time : SRTF
Lowest average response time: RR_q2
Gantt segments written to outputs/scheduling/gantt_<alg>.csv
```

Screenshot 7 – Comparison of all policies with measured arrivals

```
codespace@ubuntu-24.04: ~/OS-MiniProject-Irrigation-Advisory
Algorithm    Avg WT(ms)  Avg TAT(ms)   Avg RT(ms)  Switches    Tasks/s
FCFS             289.25       370.50       289.25         7       12.3
SJF              148.75       230.00       148.75         7       12.3
SRTF             148.75       230.00       148.75         7       12.3
RR_q2            295.75       377.00         6.25       309       12.3
RR_q5            300.62       381.88        14.50       128       12.3
RR_q10           305.00       386.25        28.25        67       12.3
```

Output 3 – Comparison for the batch cycle (all tasks queued at t = 0)



Figure 6 – Average waiting, turnaround and response time (measured scenario)

Analysis and justification

- **SJF/SRTF minimise average waiting time.** In the batch cycle, average WT drops from 289.2 ms (FCFS) to 148.8 ms (SJF), a 49 % reduction. With measured arrivals it drops from 221.2 ms to 158.1 ms (SJF, 29 %) and 122.8 ms (SRTF, 45 %). The short tasks (weather acquisition, report generation) no longer wait behind the 176 ms Sameerwadi prediction.
- **RR gives the best response time but the worst waiting time.** With q = 2 ms the average response time is only 3.2 ms, but there are 294 context switches and average WT rises to 251.0 ms. Larger quanta (5 and 10 ms) cut the switches to 123 and 65 but increase response time. As q grows, RR behaves more like FCFS.

- **Best policy for this use case.** The advisory cycle is a **batch job with known, repeatable burst times**: each task's CPU time is proportional to its village's plot count and is measured every run. This is the ideal case for **SJF**, because the main weakness of SJF (unknown burst lengths) does not apply here. SJF delivers the most advisories early, so they are ready before the morning power window when farmers start their pumps. SRTF is marginally better but adds preemption overhead. For interactive work on the same gateway, such as farmer queries or dashboard requests, RR with q ≈ 5–10 ms keeps response time under about 30 ms at a moderate switch cost. A practical design is to run the batch tasks with SJF and the interactive tasks with RR.
- Starvation risk: Sameerwadi, the largest village, always finishes last under SJF. With only 8 tasks per cycle this is bounded (it finishes in under 0.7 s). Ageing can be added if more villages are onboarded.

Post-lab questions

Q1. Why is SJF optimal for average waiting time, and why is it hard to use in practice?
Running the shortest job first minimises the sum of waiting times, which an exchange argument proves. The difficulty is that burst lengths are usually unknown. In our use case they are measured each cycle, so SJF is practical here.

Q2. How does the time quantum affect Round Robin?
A very small q gives good responsiveness but many context switches (294 at q = 2). A very large q makes RR degrade into FCFS. q should be somewhat larger than a typical short burst.

Q3. What is the difference between SJF and SRTF?
SRTF is preemptive: a newly arrived job with a shorter remaining time preempts the running one. In the measured scenario this lowers average WT from 158.1 to 122.8 ms.

Outcome / conclusion. Scheduling the prototype's real, measured processes shows that SJF suits the periodic advisory batch and RR suits interactive requests. The choice is backed by the measured WT, TAT, response-time and context-switch figures.

7. Experiment 7 – Producer–Consumer using Semaphores and Mutex

Aim. Apply a classical synchronisation problem to a shared resource of the use case and show evidence that race conditions are prevented.

Use-case connection. In the field, several village sensor gateways push soil-moisture readings to an edge server while analytics workers process them. `bin/sensor_sync` models this exactly:

Role	Implementation
Producers	5 threads, one per village gateway, streaming the 100,277 well-formed readings from sensor_readings.csv
Shared buffer	Bounded circular buffer of 8 readings
Consumers	3 analytics threads that validate each reading and update the **shared plot-water table** (per-plot count and moisture sum, plus global totals)
Synchronisation	`sem_t empty` (free slots, initial value 8): the producer blocks when the buffer is FULL. `sem_t full` (filled slots, initial value 0): the consumer blocks when the buffer is EMPTY. `buf_mutex` protects the in/out indices. `table_mutex` protects the plot table. Poison-pill items shut the consumers down cleanly.

```
/* producer */                          /* consumer */
sem_wait(&empty);                       sem_wait(&full);
pthread_mutex_lock(&buf_mutex);         pthread_mutex_lock(&buf_mutex);
buf[in] = r; in = (in+1) % N;           r = buf[out]; out = (out+1) % N;
pthread_mutex_unlock(&buf_mutex);       pthread_mutex_unlock(&buf_mutex);
sem_post(&full);                        sem_post(&empty);
                                        pthread_mutex_lock(&table_mutex);   /* removed by --unsafe */
                                        plot_count[p]++; plot_sum[p] += sm; total_valid++;
                                        pthread_mutex_unlock(&table_mutex);
```

```
./bin/sensor_sync            # safe: semaphores + mutex
./bin/sensor_sync --unsafe   # table_mutex removed -> race condition
./bin/sensor_sync -b 1 -p 8 -c 8   # stress test: buffer always full/empty
```

```
codespace@ubuntu-24.04:~/OS-MiniProject$ ./bin/sensor_sync
EXPERIMENT 7 - Producer-Consumer on the sensor buffer (KJS-AGR-01)
mode=SAFE (semaphores + mutex)  producers=5 (village gateways)  consumers=3 (analytics workers)  buffer=8 slots
readings to stream: 100277

check                                 expected       actual result
readings produced = consumed            100277       100277 OK
valid readings counted                   99258        99258 OK
rejected readings counted                 1019         1019 OK
per-plot table entries                   99258        99258 OK
moisture sum (x100)                  253335969    253335969 OK

buffer FULL events (producer blocked) : 20527
buffer EMPTY events (consumer blocked): 3469
elapsed 185.1 ms, throughput 541648 readings/s

RESULT: shared plot table is CONSISTENT - no lost updates, no race condition.
```

Screenshot 8 – SAFE run: every check matches the sequential reference, so no updates are lost

```
codespace@ubuntu-24.04:~/OS-MiniProject$ ./bin/sensor_sync --unsafe
EXPERIMENT 7 - Producer-Consumer on the sensor buffer (KJS-AGR-01)
mode=UNSAFE (no table mutex)  producers=5 (village gateways)  consumers=3 (analytics workers)  buffer=8 slots
readings to stream: 100277

check                                  expected       actual result
readings produced = consumed             100277       100277 OK
valid readings counted                    99258        36898 MISMATCH  ⟵ race condition
rejected readings counted                  1019         1009 MISMATCH  ⟵ race condition
per-plot table entries                    99258        69698 MISMATCH  ⟵ race condition
moisture sum (x100)                   253335969    177674255 MISMATCH  ⟵ race condition

buffer FULL events (producer blocked) : 66342
buffer EMPTY events (consumer blocked): 32
elapsed 775.3 ms, throughput 129339 readings/s

RESULT: 62370 updates LOST - unsynchronised read-modify-write caused a RACE CONDITION.
```

Screenshot 9 – UNSAFE run (no table mutex): 62,370 updates lost, which proves the race condition

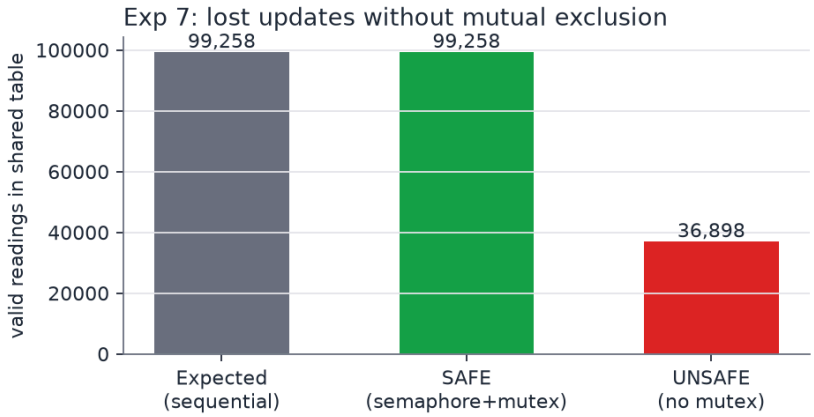


Figure 7 – Valid readings recorded in the shared table: the safe run matches exactly, the unsafe run loses 63 %

Results

- SAFE: produced = consumed = 100,277, and the valid count, rejected count, per-plot entries and moisture checksum all equal the single-threaded reference. Producers blocked on a FULL buffer 20,527 times and consumers blocked on an EMPTY buffer 3,469 times, which shows that the semaphores actually blocked threads. Throughput was about 541 k readings/s.
- UNSAFE: with the mutex removed, two consumers read the same old counter value and both write back value + 1, so one increment is lost. The table recorded only 36,898 of 99,258 valid readings. Note that produced = consumed still holds, because the buffer is still protected by its semaphores. The lost updates are confined to the unprotected table.
- The buffer-size-1 stress test (8 producers, 8 consumers) is also consistent (Section 9).

Post-lab questions

Q1. Why are two semaphores and a mutex needed?
The counting semaphores track free and filled slots, so threads block instead of busy-waiting. The mutex gives mutual exclusion on the shared indices and table. Semaphores alone do not stop two consumers from updating the table at the same time.

Q2. What happens if the order of sem_wait(&empty) and mutex_lock is swapped in the producer?
A producer could hold the mutex while it sleeps on a full buffer. The consumer then cannot acquire the mutex to free a slot, so the system deadlocks.

Q3. What is a race condition?
The result depends on the timing of the interleaving of concurrent accesses to shared data. Here, an unprotected read-modify-write of a counter loses updates.

Outcome / conclusion. The producer–consumer solution lets concurrent sensor gateways and analytics workers share the buffer and the plot table safely. The unsafe run gives direct, measured evidence of what goes wrong without synchronisation.

8. Experiment 9 – Deadlock Avoidance using the Banker's Algorithm

Aim. Identify the processes and shared resources of the use case and apply the Banker's safety and resource-request algorithms to safe and unsafe scenarios.

Use-case connection. Within the next 48-hour irrigation window, the five villages (processes) compete for four types of shared resource managed by the cane department:

Resource	Meaning	Total instances
Pumps	shared mobile pump sets	14
Power	3-phase feeder slots of 15 kW during the power window	6
Valves	canal outlet gates	10
FertUnits	drip fertigation units	6

Each village's **maximum claim** is the demand computed by the advisory stage of Experiment 4 (`outputs/pump_demand.csv`). The values are fixed in `data/config/banker_scenario.txt` so that the demonstration is reproducible. A deadlock here would mean, for example, that Sameerwadi holds pumps and waits for power slots while Mudhol holds power slots and waits for pumps, so neither can irrigate.

codespace@ubuntu-24.04: ~/OS-MiniProject-Irrigation-Advisory

```
village,plots_today,plots_tomorrow,plots_48h,pumps,power_blocks,valves,fertigation_units,water_m3
Sameerwadi,68,27,95,8,3,5,4,290090
Mudhol,57,24,81,7,3,5,3,222203
Jamkhandi,34,16,50,5,2,3,2,116297
Rabkavi,51,18,69,6,3,4,3,186223
Terdal,34,10,44,4,2,3,1,137548
```

Output 4 – outputs/pump_demand.csv from the advisory stage: the source of the maximum claims

```
./bin/banker                                  # runs data/config/banker_scenario.txt
```

```
codespace@ubuntu-24.04:~/OS-MiniProject$ ./bin/banker | head -42
EXPERIMENT 9 - Banker's algorithm for irrigation resources (KJS-AGR-01)
scenario: data/config/banker_scenario.txt

Resources: Pumps, Power, Valves, FertUnits
Total    : [14 6 10 6]

Job          Allocation     Max            Need
Sameerwadi   [3 1 2 2]     [8 3 5 4]     [5 2 3 2]
Mudhol       [2 1 2 1]     [7 3 5 3]     [5 2 3 2]
Jamkhandi    [2 0 1 1]     [5 2 3 2]     [3 2 2 1]
Rabkavi      [3 1 1 1]     [6 3 4 3]     [3 2 3 2]
Terdal       [1 1 1 0]     [4 2 3 1]     [3 1 2 1]
Available    [3 2 3 1]

INITIAL STATE - safety algorithm:
  Jamkhandi    Need [3 2 2 1] ≤ Work [3 2 3 1]  → finishes, Work = [5 2 4 2]
  Rabkavi      Need [3 2 3 2] ≤ Work [5 2 4 2]  → finishes, Work = [8 3 5 3]
  Terdal       Need [3 1 2 1] ≤ Work [8 3 5 3]  → finishes, Work = [9 4 6 3]
  Sameerwadi   Need [5 2 3 2] ≤ Work [9 4 6 3]  → finishes, Work = [12 5 8 5]
  Mudhol       Need [5 2 3 2] ≤ Work [12 5 8 5]  → finishes, Work = [14 6 10 6]
  SAFE. Safe sequence: < Jamkhandi, Rabkavi, Terdal, Sameerwadi, Mudhol >

REQUEST Terdal [1 1 0 0]
  pretend allocation, run safety algorithm:
  Terdal       Need [2 0 2 1] ≤ Work [2 1 3 1]  → finishes, Work = [4 3 4 1]
  Jamkhandi    Need [3 2 2 1] ≤ Work [4 3 4 1]  → finishes, Work = [6 3 5 2]
  Rabkavi      Need [3 2 3 2] ≤ Work [6 3 5 2]  → finishes, Work = [9 4 6 3]
  Sameerwadi   Need [5 2 3 2] ≤ Work [9 4 6 3]  → finishes, Work = [12 5 8 5]
  Mudhol       Need [5 2 3 2] ≤ Work [12 5 8 5]  → finishes, Work = [14 6 10 6]
  SAFE. Safe sequence: < Terdal, Jamkhandi, Rabkavi, Sameerwadi, Mudhol >
  ⇒ GRANTED. Available now [2 1 3 1]

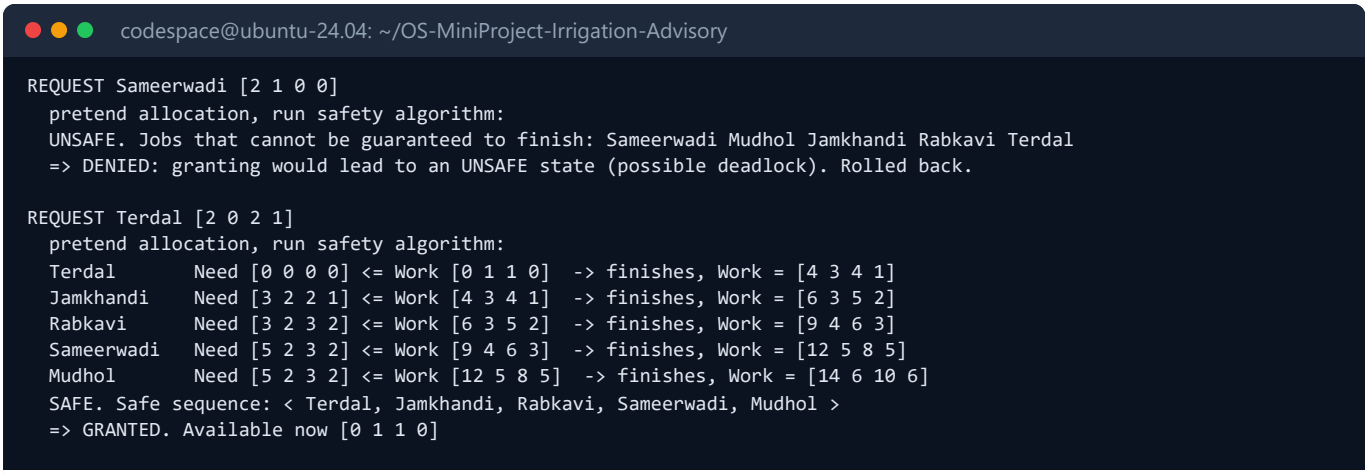
REQUEST Sameerwadi [2 1 0 0]
  pretend allocation, run safety algorithm:
  UNSAFE. Jobs that cannot be guaranteed to finish: Sameerwadi Mudhol Jamkhandi Rabkavi Terdal
  ⇒ DENIED: granting would lead to an UNSAFE state (possible deadlock). Rolled back.

REQUEST Terdal [2 0 2 1]
  pretend allocation, run safety algorithm:
  Terdal       Need [0 0 0 0] ≤ Work [0 1 1 0]  → finishes, Work = [4 3 4 1]
  Jamkhandi    Need [3 2 2 1] ≤ Work [4 3 4 1]  → finishes, Work = [6 3 5 2]
  Rabkavi      Need [3 2 3 2] ≤ Work [6 3 5 2]  → finishes, Work = [9 4 6 3]
```

Screenshot 10 – Initial state (Allocation, Max, Need, Available), the safety algorithm, and the first request being granted

```
codespace@ubuntu-24.04:~/OS-MiniProject$ ./bin/banker | grep -E "^REQUEST|^RELEASE⟹|ERROR|WAIT|DENIED"
REQUEST Terdal [1 1 0 0]
  ⇒ GRANTED. Available now [2 1 3 1]
REQUEST Sameerwadi [2 1 0 0]
  ⇒ DENIED: granting would lead to an UNSAFE state (possible deadlock). Rolled back.
REQUEST Terdal [2 0 2 1]
  ⇒ GRANTED. Available now [0 1 1 0]
REQUEST Mudhol [6 0 0 0]
  ERROR: request exceeds declared maximum claim for Pumps (need 5) → REJECTED
REQUEST Jamkhandi [1 0 0 0]
  only 0 Pumps available → job must WAIT (not granted)
RELEASE Terdal: irrigation complete, returns [4 2 3 1]  → Available [4 3 4 1]
REQUEST Jamkhandi [1 0 0 0]
  ⇒ GRANTED. Available now [3 3 4 1]
```

Screenshot 11 – Decision for every request: GRANTED, DENIED (unsafe), GRANTED, ERROR, WAIT, RELEASE, GRANTED



Output 5 – Detail: Sameerwadi's request leaves no job able to finish, so it is denied. Terdal's full-claim request is granted

Results

#	Event	Outcome	Reason
0	Initial state	SAFE	Safe sequence: Jamkhandi → Rabkavi → Terdal → Sameerwadi → Mudhol
1	Terdal requests [1 1 0 0]	GRANTED	The state remains safe
2	Sameerwadi requests [2 1 0 0]	DENIED	It would take the last pumps and power, leaving no job able to finish (unsafe → possible deadlock), so the request is rolled back
3	Terdal requests its full remaining need [2 0 2 1]	GRANTED	Terdal can complete and release its resources
4	Mudhol requests 6 pumps	ERROR	This exceeds Mudhol's declared need of 5
5	Jamkhandi requests 1 pump	WAIT	0 pumps available
6	Terdal finishes irrigating	RELEASE	[4 2 3 1] is returned
7	Jamkhandi retries	GRANTED	Safe after the release

Post-lab questions

Q1. What is a safe state? Is an unsafe state always a deadlock?
A state is safe if there is some order in which every process can obtain its remaining need and finish. An unsafe state is not necessarily a deadlock, but the system can no longer guarantee to avoid one. The Banker's algorithm therefore never enters an unsafe state (request 2).

Q2. Which of the four Coffman conditions hold in this use case?
All four can hold. A pump serves one plot at a time (mutual exclusion). A village holds pumps while waiting for power (hold and wait). Pumps cannot be taken away mid-irrigation (no preemption). Circular wait can form between villages. Banker's avoids deadlock by refusing requests that could lead to circular wait.

Q3. What are the limitations of the Banker's algorithm here?
Maximum claims must be declared in advance. Our advisory stage computes them, which is realistic. The number of processes is assumed fixed, and the safety check costs $O(m \cdot n^2)$ per request, which is trivial for 5 villages and 4 resource types.

Outcome / conclusion. Treating irrigation resources as OS resources lets the Banker's algorithm produce a deadlock-free pump and power allocation plan. The connection to the application is direct, because the

maximum claims come from the AI advisory's demand.

9. Testing and Performance Comparison

`tests/run_tests.sh` (also run in GitHub Actions CI on every push) runs normal and failure scenarios for all four experiments and checks exit codes and output text.

```
codespace@ubuntu-24.04:~/OS-MiniProject$ bash tests/run_tests.sh
== Exp 4: pipeline (fork/exec/wait, file system calls) ==
  PASS  normal run on synthetic data                                (exit 0)
  PASS  missing sensor file → stage 1 fails, workflow aborts        (exit 1)
  PASS  corrupt sensor file → acquire exits 3                       (exit 3)
  PASS  missing model → rule-based fallback warning
  PASS  unknown village → predict exits 3                           (exit 3)
== Exp 6: scheduler ==
  PASS  measured bursts, quanta 2,5,10                              (exit 0)
  PASS  missing stage log → falls back to documented estimates
  PASS  invalid quantum 0 rejected                                  (exit 1)
== Exp 7: producer-consumer ==
  PASS  safe mode, buffer 8 → consistent table                      (exit 0)
  PASS  safe mode, buffer 1 (always full/empty) → consistent        (exit 0)
  PASS  unsafe mode → race detected (exit 4)                        (exit 4)
  PASS  invalid buffer size 0 rejected                              (exit 1)
== Exp 9: Banker's algorithm ==
  PASS  initial state is safe
  PASS  unsafe request is denied
  PASS  request above max claim is rejected
  PASS  missing scenario file                                       (exit 2)

Tests passed: 16   failed: 0
```

Screenshot 12 – 16 / 16 normal and failure-scenario tests pass on Ubuntu 24.04

Failure scenario	Expected behaviour	Observed
Sensor data file missing	Stage 1 child exits 2, parent aborts and later stages never run	Pass (exit 1)
Corrupt sensor file (no valid row)	acquire exits 3	Pass
Model file missing	Warning and automatic rule-based fallback	Pass
Unknown village / no forecast	predict exits 3	Pass
Stage-time log missing	Scheduler uses documented estimates and says so	Pass
Invalid time quantum (0)	Rejected with an error	Pass
Buffer of size 1 (always full/empty)	Still consistent	Pass
Missing mutex (--unsafe)	Race detected (exit 4)	Pass
Unsafe / over-claim / unavailable resource request	DENIED / ERROR / WAIT	Pass

Measurements summary

Metric	Value
Depth prediction error (ML vs rule)	MAE 3.51 mm vs 24.93 mm · R^2 0.986 vs 0.430
Stress classifier (ML vs rule)	Accuracy 0.861 vs 0.858 · F1 0.857 vs 0.855
Pipeline execution time (600 plots, 8 processes)	376 ms wall, 650 ms total child CPU (2 vCPU)
Sensor data quality	99,258 valid, 1,542 rejected (1.53 %)
Estimated water saving vs rule schedule	34.6 % (3-day requirement)
Scheduling, average WT (batch cycle)	FCFS 289.2 · SJF 148.8 · SRTF 148.8 · RR(q=2/5/10) 295.8 / 300.6 / 305.0 ms
Scheduling, average WT (measured arrivals)	FCFS 221.2 · SJF 158.1 · SRTF 122.8 · RR(q=2/5/10) 251.0 / 256.5 / 260.2 ms
Producer–consumer throughput	≈ 541,648 readings/s (safe) · 62,370 updates lost when unsafe
Banker's decisions	3 granted, 1 denied (unsafe), 1 error, 1 wait

> Timings vary slightly between runs because they are measured on a shared cloud VM. The values in this report are from the run whose full logs are in `report/evidence/`.

10. Limitations, Data Privacy and Improvements

Limitations.

- The data is synthetic and the labels come from a physically based simulation, so the model accuracy shows the method works but not its field accuracy. The model must be retrained and validated on KIAAR data before any real use.
- The models are linear or logistic and use an assumed pump flow (10 m^3/h per kW) and irrigation efficiencies (drip 0.9, furrow 0.6).
- The scheduler is a simulation using measured bursts. The real Linux CFS scheduler ran the actual processes. The Banker's maximum claims are fixed per window.

Data privacy. Farmer identities are anonymised tokens. No names, phone numbers or exact boundaries are stored. Output files are created with mode 0640 and backups are read-only (0440). A deployment would need encryption, role-based access and DPDP-Act-compliant consent, as listed in the use-case governance table. Advisories are decision support: the farmer or field officer keeps the final decision (human in the loop).

Possible improvements. Train gradient-boosted or LSTM models on real sensor history. Add an LLM layer for Kannada and Marathi advisories and a chatbot. Use shared memory and message queues between stages. Add an MQTT-based IoT ingestion service. Use ageing in SJF. Recompute the Banker's claims dynamically every advisory cycle. Add a web dashboard with a map of the plots.

11. Conclusion

We built a working Linux prototype of the KJS-AGR-01 irrigation advisory system with a clear input → processing → output flow. It turns raw sensor, weather and plot data into plot-wise, prioritised irrigation advisories using a trained ML model. Four OS experiments are built into the application rather than shown as separate demos. Processes and system calls run the workflow (Exp 4). Its measured bursts drive the scheduling study (Exp 6). Semaphores and a mutex protect the shared sensor buffer and plot table (Exp 7). The Banker's algorithm allocates the pumps and power that the advisory asks for without deadlock (Exp 9).

References

1. KJS-AGR-01 AI Use Case Profile – Irrigation Advisory System for Sugarcane, K J Somaiya (Dr. Shyamal Virnodkar et al.).
2. Operating Systems Experiments through Somaiya Vidyavihar AI Use Cases – assignment document.
3. Allen R. G., Pereira L. S., Raes D., Smith M., *FAO Irrigation and Drainage Paper 56: Crop Evapotranspiration*, FAO, 1998.
4. Hargreaves G. H., Samani Z. A., "Reference crop evapotranspiration from temperature", *Applied Engineering in Agriculture*, 1985.
5. Silberschatz A., Galvin P. B., Gagne G., *Operating System Concepts*, 10th ed., Wiley.
6. Linux man-pages: fork(2), execve(2), wait4(2), open(2), read(2), write(2), sem_init(3), pthread_mutex_lock(3).

Acknowledgement of AI-Assisted Work

As permitted by the assignment, AI tools were used for assistance with coding and documentation. All code was compiled, executed and verified by the team on Ubuntu Linux. Every output, timing and screenshot in this report comes from real runs (full logs in `report/evidence/`).